Guion de los expositores — versión de 15 minutos

A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting (Euro-Par 2024)

Reparto y tiempos

Expositor	Diapositivas	Tiempo
David — el problema	1–9	≈ 5 min
Hugo — la solución	10–16	≈ 5 min
Diego — corrección, evaluación y conclusiones	17–24	≈ 5 min
Preguntas	25 (26 es de reserva)	resto

Archivo: `exposicion_15min.pdf` (26 diapositivas). Ritmo: unos 35 s por diapositiva; las dos demos (8 y 16) llevan 45 s. Si vais justos de tiempo, la diapositiva que se puede acortar sin perder ideas es la 7 (garantías, una frase) y la 19 (solo leer los números en negrita).

Cómo se ha condensado (para que no falte ningún concepto)

Versión de 30 min	En la de 15 min
Plan, portadillas de parte	Se quitan; el pie indica «Expone: Nombre»
Conteo de referencias + shared/atomic shared pointer	Una sola diapositiva (4)
Lo que había, Contribuciones	Se dicen de palabra al final de la 2 y en la 24
Peso en el atomic shared pointer + Empaquetar en 64 bits	Una sola (11)
Recargar el talonario + Figura 4 (idea informal)	Una sola (14)
Ejemplo completo P = 4, C = 8	Fuera; está en `exposicion.pdf` (diap. 24) por si preguntan
Demo 2 (tabla + traza)	Una sola (16), con el contraejemplo contado de palabra
Coste + Extensiones, Fig. 5 + Fig. 6 + montaje	Dos diapositivas (18, 19)
¿Coincide con el paper?, split reference count	Sus matices pasan a las diap. 21–22 y 23

Antes de empezar

- Abrir el PDF a pantalla completa. Si queréis repetir las demos: `bash codigo/ejecutar_pruebas.sh` (desde `codigo/`) y `node simulador/buscar.js 20000 ingenua,sinReserva` (desde `simulador/`). Si falla, enseñad la captura.

- Analogía: **libro** = objeto, **hoja** = contador, **pase** = referencia (peso 1), **cartel** = *atomic shared pointer*, **talonario** = peso, **tirar el libro** = liberar.

Parte 1 — David (≈ 5 min)

1. Portada (10 s) — «Exponemos un paper de Euro-Par 2024 sobre conteo de referencias concurrente. Yo el problema, Hugo la solución y Diego la evaluación.»

2. El paper en una frase (30 s)

- La pregunta: leer un puntero compartido con conteo **mientras otro hilo lo cambia**, sin candados, sin esperas y sin memoria extra.
- Mejora el *split reference count* (Folly, GCC), solo *lock-free*; el paper lo hace *wait-free* y más simple. Hay código público (sello *Artifact Evaluated*) y lo usamos.

3. Analogía (40 s) — Recorrer la tabla: libro, hoja, pase, cartel, talonario. Frase: «el cartel lleva un talonario de tickets ya apuntados en la hoja; leer es arrancar un ticket».

4. Conteo de referencias y atomic shared pointer (40 s)

- Copiar suma 1, destruir resta 1, a 0 se libera en ese momento; con rc = 1 se modifica en el sitio.
- Shared pointer = inmutable; atomic shared pointer = celda mutable con `load`/`store` (`std::atomic<std::shared_ptr<T>>`). El problema nace al leer esa celda.

5. La carrera read-reclaim (50 s) — Seguir las flechas: Ana lee A ⟶ la pausan ⟶ Beto pone B, resta uno a A (rc = 0) y **libera A** ⟶ Ana suma 1 a un libro que ya no existe. Son dos pasos no atómicos; la ventana dura nanosegundos, por eso aparece en producción.

6. ¿Por qué no en un solo paso? (35 s) — Puntero y contador viven en dos sitios. Existen `fetch_and_add`, `swap`, CAS y CAS de doble anchura (palabras contiguas); no existen DCAS ni `fetch_and_add` de 128 bits. Una atómica abarca una palabra.

7. Garantías de progreso (35 s) — Bloqueante: alguien puede esperar siempre. *Lock-free*: siempre avanza *algún* hilo (bucle CAS, yo podría fallar siempre). *Wait-free*: **cada** hilo acaba en un número acotado de pasos propios. Importa porque la gestión de memoria está en todas partes.

8. Demo 1: AddressSanitizer (45 s)

- Arriba, ingenua con 8 hilos: señalar `heap-use-after-free`, el `WRITE` en `Ingenua::load()` (el `fetch_add`) y `freed by thread T2 … Ingenua::store()`: justo lo de la diap. 5.
- Abajo, código de los autores en las mismas condiciones: 1 600 000 operaciones, 56 ms, sin errores.

9. La idea (35 s) — Cartel = (libro A, talonario 104); hoja de A = 106 = 104 + 1 + 1. Los tickets ya están pagados; leer = un `fetch_and_add` que arranca un ticket y dice qué libro hay. Leer y apuntarse son **un solo paso**. **Transición:** «Cómo se mantiene esa cuenta correcta lo explica Hugo.»

Parte 2 — Hugo (≈ 5 min)

10. Conteo ponderado (40 s) — Cada referencia tiene un peso; el contador es la suma. Clonar reparte el peso **sin tocar el contador**. Tabla: X = 10 ⟶ clonar (6 + 4, hoja no se toca) ⟶ tirar Y (hoja 6) ⟶ … ⟶ hoja 0, se libera. Evitar tocar la hoja elimina la carrera.

11. El peso en el atomic shared pointer (45 s)

- Fig. 2: (a) cartel 104, dos pases, hoja 106; (b) tras un `load`, cartel 103 y un pase nuevo, hoja sigue en 106.
- Al crear: peso C ≫ P, se suma C a la hoja. Invariante: hoja ≥ peso + pases.
- Empaquetado: peso en los 16 bits altos libres; restar un ticket = restar 2^{48}; la dirección no cambia.

12. Algoritmo 1 (50 s) — `store`: sumar C al nuevo *antes* de publicar; `swap` de golpe; restar al viejo **solo el sobrante** w. `load`: un `fetch_and_add` de −1 ticket; si quedan menos de P, recargar; devolver un pase ya pagado. El viejo se libera cuando le toca, no se retrasa.

13. Por qué desaparece la carrera (45 s) — Un `fetch_and_add` (F) y un `swap` (S2) tocan la misma palabra: solo hay dos órdenes. Caso 1: F antes ⟶ Ana tiene pase de A y la hoja deja 1 para ella. Caso 2: F después ⟶ Ana se lleva un pase de B y no toca A. No hay tercer caso.

14. Recargar el talonario (55 s) — Si el peso baja de P, ese hilo recarga: reserva C − w y sube el peso con un **CAS** (el cartel pudo cambiar de libro o alguien recargar ya). R-1: C ≥ 2P. R-2: el peso solo baja de 1 en 1 o se fija por encima de P. Fig. 4: si el CAS falla y el peso subió, otro ya recargó ⟶ **salida temprana**.

15. Algoritmo 2 (40 s) — Reservar C − w **antes** del CAS; éxito ⟶ terminar; otro puntero o peso mayor ⟶ devolver la reserva y salir; peso menor ⟶ reintentar. Los pesos esperados forman una cadena estrictamente decreciente entre P−1 y 0: como mucho P intentos.

16. Demo 2: el simulador (45 s) — Simulador de intercalados (20 000 ejecuciones por variante). Señalar la fila **Paper: 0 fallos**; las variantes rotas (ingenua 4 796, umbral 0: 5 089, sin reserva 1 171, sin salida temprana 5 467) fallan en 2 a 7 pasos. Contar el contraejemplo de «sin reserva»: Carla hace `load`, Beto hace `swap` y resta toda la reserva ⟶ liberación con un lector dentro. Abajo: con 4 hilos, intentos de CAS 1: 80,1 %, 2: 18,2 %, 3: 1,7 %, 4: 0,06 % (cota P = 4). **Transición:** «Eso es evidencia; la demostración y la evaluación las explica Diego.»

Parte 3 — Diego (≈ 5 min)

17. La demostración (40 s) — El peso es de 16 bits sin signo: −1 sería 65 535 tickets fantasma. Ofensa = decrementar con peso ≤ P; O = multiconjunto de ofensores. Invariante 1: cada hilo está en O como mucho una vez. Invariante 2: peso ≥ P − |O|. Luego peso ≥ P − P = 0. Por eso el umbral es P y no 0.

18. Coste y extensiones (40 s) — Bucle CAS ≤ P intentos. `load` suele ser una instrucción; **O(P) es el peor caso**. Con C = 2^{19} y P = 2^{16} hay ≈ 458 752 lecturas entre recargas. `null`, punteros débiles y *aliasing*; compare-and-swap y `load` *aliased* solo *lock-free*.

19. Experimentos del paper (45 s) — Pila concurrente, 144 hilos (rendimiento) y 56 (latencia). Con 1 hilo la más rápida; ≈ 7× Williams y ≈ 20× Folly con 1 pila y 144 hilos; pierde frente a Anderson con una sola pila (CAS *lock-free*). `load`, p = 56: Ours 1,14 µs; Anderson 2,58; Folly 11,6; Libstdc++ 99,7.

20. Pruebas en local (1): el boceto (40 s) — P = 8, C = 16. Versión del paper: 0 errores y máx. 7 intentos (≤ P). Ingenua: 601 681 errores y −102 892 «sin liberar» (dobles liberaciones). Sin salida temprana: 0 errores pero 38 intentos > P (no *wait-free*); con C = P: 50 060 errores.

21. Pruebas en local (2): rendimiento (40 s) — i5-10400, 12 hilos, WSL. n = 1, u = 1: Autores 34,3 · Libstdc++ 2,2 · Mutex 6,4 Mop/s. **Matiz honesto**: con u = 50 el mutex gana (2,9 frente a 2,2) porque el CAS reintenta. Con 1 hilo, Autores es la más rápida.

22. Pruebas en local (3): latencia (40 s) — `load` (mediana / p99): Autores 269 ns / 0,83 µs; Libstdc++ 3,06 / 16 µs; Mutex 61 ns / **27,6 µs**. La mediana engaña; la cola delata a quien espera.

23. Valoración crítica (30 s) — Fortalezas: simple, *wait-free*, sin memoria extra, liberación inmediata, lecturas de una instrucción. Límites: CAS y *aliasing lock-free*; cada lectura **escribe**; límite de P hilos (de corrección); depende de bits libres del puntero.

24. Conclusiones (40 s) — (1) *Wait-free* sin retrasar la liberación; (2) peso prepagado en el puntero; (3) recarga ≤ P intentos por R-1 y R-2; (4) comprobado en local. Dejar las dos preguntas de discusión.

Preguntas probables (respuesta corta)

Para David

1. *¿Por qué no un candado?* Es bloqueante: un hilo parado con el candado frena a todos (mutex: p99 de 27,6 µs).
2. *¿Por qué no un DCAS?* Hay que modificar dos posiciones separadas a la vez y el hardware actual no lo tiene.
3. *¿Con qué frecuencia ocurre la carrera?* Rarísima (nanosegundos), pero con ASan y 8 hilos la ingenua cae enseguida.
4. *¿Lock-free frente a wait-free?* Lock-free: siempre avanza *algún* hilo. Wait-free: **cada** hilo acaba en pasos acotados.
5. *¿Por qué no punteros de riesgo?* Gastan memoria extra y retrasan la liberación.

Para Hugo

1. *¿Qué es el peso?* El talonario del cartel: tickets ya apuntados en la hoja.
2. *¿Por qué C ≥ 2P?* Tras recargar el peso queda ≥ P + 1 aun con P ofensores; con C = P el boceto dio 50 060 errores.
3. *¿Para qué la salida temprana?* Si el peso subió, otro ya recargó; sin ella hubo 38 intentos con P = 8.
4. *¿Y si el cartel cambia de libro?* El CAS compara también el puntero: falla, se devuelve la reserva y se sale.
5. *¿Caben puntero y peso en 64 bits?* Sí: 48 bits de dirección y 16 libres en x86-64.

Para Diego

1. *¿Cuánto cuesta un `load`?* Normalmente una instrucción; O(P) solo en el peor caso.
2. *¿Por qué el umbral es P?* Es el colchón: peso $\geq P - |O| \geq 0$.
3. *¿Y con más de P hilos?* Se pierde la corrección; P es el máximo de hilos por variable.
4. *¿Por qué no 144 hilos?* Nuestro equipo tiene 12; coinciden las tendencias, no las cifras absolutas.
5. *¿Por qué a veces gana el mutex?* Con 1 pila y 50 % de actualizaciones el CAS *lock-free* reintenta; el paper lo reconoce.

Errores típicos que evitar

- «El algoritmo retrasa la liberación»: **falso**; libera al llegar a 0.
- «Es O(P) siempre»: O(P) es el **peor caso**.
- «Todo es wait-free»: solo `load`, `store` y `swap`; CAS y *aliasing* son *lock-free*.
- Presentar los 144 hilos como medidas nuestras: las nuestras son de 12 hilos.
- «El mutex es peor en todo»: gana con u = 50 y en la mediana de `load`; pierde en la cola.
- Dar cifras sin contexto (hilos, n, u): especialmente los 34,3 Mop/s.